To make Bina.ai a true, enterprise-grade platform, the **Widget Library** must be domain-agnostic. It should allow an Architect to build an app for *any* field—whether it's a rural farming assistant, a local legal-rights advisor, an offline inventory tracker, or a dialect-translation tool.

Here is the complete, expanded Bina.ai Widget Library. It takes the "drag-and-drop" logic of AWS PartyRock but completely re-engineers it for **mobile-first, 100% offline, LiteRT-powered** execution.

📥 Category 1: Sensory & Input Widgets (Capturing the World)

These widgets gather unstructured data using the device's physical hardware and local edge processing.

1. `VoiceText_Transcriber` (Edge Speech-to-Text)

- **Purpose:** The primary input for low-literacy users or hands-free environments. Uses on-device models to transcribe audio to text.
- **Architect Settings:**
  - `input_mode`: [Tap-to-Speak, Hold-to-Speak]
  - `primary_language_model`: [Standard Malay, English, Local Dialect Pack]
  - `fallback_text_input`: Boolean (Allows typing if the user prefers).
- **Cross-Domain Uses:** A mechanic describing a broken engine; a farmer logging daily egg counts; a user asking a legal question.

2. `Vision_EdgeScanner` (Local Camera Processing)

- **Purpose:** Replaces the desktop "File Upload." Taps into the camera to process visual data entirely offline via LiteRT vision models.
- **Architect Settings:**
  - `inference_mode`: [OCR (Extract Text), Object_Classification, Barcode/QR_Scan]
  - `target_domain`: [General, Agriculture, Documentation, Medical_Basic]
  - `auto_crop`: Boolean (Automatically focuses on the document or object).
- **Cross-Domain Uses:** Scanning handwritten receipts (Finance); taking photos of pests (Agriculture); scanning an ID card for offline event registration (Admin).

3. `Geo_Locator` (Offline GPS Node)

- **Purpose:** Grabs the user's exact offline coordinates from the phone's GPS chip without needing a network ping.
- **Architect Settings:**
  - `precision`: [High (Exact Pin), Low (General Area)]
  - `trigger_event`: [On_App_Open, On_Button_Tap]

- **Cross-Domain Uses:** Tagging the location of a broken water pipe (Infrastructure); verifying the delivery location of aid (Logistics).

🖐️ Category 2: Tactical Widgets (Structured Manual Input)

These widgets replace complex web forms with massive, mobile-friendly touch targets.

4. `Macro_Grid` (High-Speed Touch UI)

- **Purpose:** Generates a highly customizable grid of massive, single-tap buttons for rapid data entry or multiple-choice selections.
- **Architect Settings:**
  - `grid_layout`: [1x2, 2x2, 3x3, Scrolling_List]
  - `buttons`: Array of `{"label": "...", "color": "...", "payload_value": "...", "icon": "..."}`
  - `allow_multi_select`: Boolean.
- **Cross-Domain Uses:** A POS system menu (Retail); a list of multiple-choice answers for a quiz (Education); a daily mood tracker (Mental Health).

5. `Analog_Context_Slider`

- **Purpose:** Captures subjective or continuous data without requiring the user to type a number.
- **Architect Settings:**
  - `range`: [e.g., 0_to_100, 1_to_10]
  - `left_label` / `right_label`: (e.g., "Safe" to "Dangerous", or "Strongly Disagree" to "Strongly Agree")
  - `step_interval`: Integer.
- **Cross-Domain Uses:** Rating pain levels (Healthcare); estimating water levels in a river (Disaster Relief); adjusting the reading difficulty of generated text (Education).

🧠 Category 3: The Brain (AI Logic & Routing)

These widgets are hidden from the Builder's UI. They act as the invisible wiring that routes data through the local Gemma 4 model.

6. `LiteRT_Core_Inference` (The Gemma 4 Node)

- **Purpose:** The central processor. It takes inputs from the Sensory/Tactical widgets and runs them through the quantized Gemma 4 model.
- **Architect Settings:**
  - `system_prompt`: (e.g., "You are an expert mechanic. Analyze the input and provide 3 troubleshooting steps.")

    - temperature: [0.0 (Strict/Deterministic) to 0.8 (Creative/Brainstorming)]
    - input_variables: Mapping inputs to the prompt (e.g., {{VoiceText}} + {{Context_Slider}}).

7. Logic_Router (Conditional Branching)

- **Purpose:** Adds traditional programming logic (If/Then) to the AI's output without writing code.
- **Architect Settings:**
    - condition_trigger: (e.g., IF {AI_Output} contains "EMERGENCY")
    - action_path_A: Trigger Widget X (e.g., Send SMS).
    - action_path_B: Trigger Widget Y (e.g., Show Checklist).
- **Cross-Domain Uses:** If an AI identifies a scam message, route to a "Warning Screen"; if it identifies a safe message, route to a "Summary Screen."

📤 Category 4: Dynamic Output Widgets (Displaying Information)

These widgets transform standard AI text generation into interactive, highly readable mobile interfaces.

8. Markdown_Canvas (The Standard Output)

- **Purpose:** A clean, highly legible text box that supports bolding, bullet points, and simple markdown tables generated by Gemma.
- **Architect Settings:**
    - text_size: [Standard, Large, Extra_Large (Accessibility)]
    - theme: [Light, Dark, High_Contrast]

9. Dynamic_ActionList (Interactive SOPs)

- **Purpose:** Forces the AI to format its advice as a clickable to-do list. The Builder physically checks off boxes as they complete real-world tasks.
- **Architect Settings:**
    - enforce_sequential_checking: Boolean.
    - on_complete_action: (e.g., "Trigger Local Ledger to log completion").
- **Cross-Domain Uses:** Step-by-step CPR instructions (First Aid); a pre-flight maintenance checklist for a drone (Agriculture); a daily habit tracker (Lifestyle).

10. Split_State_Pane (Dual View)

- **Purpose:** Splits the screen vertically or horizontally to compare two sets of data side-by-side.
- **Architect Settings:**
    - top_pane_source: {{Variable_1}}
    - bottom_pane_source: {{Variable_2}}
- **Cross-Domain Uses:** Original legal contract vs. AI-simplified summary (Legal); Formal text vs. Dialect translation (Education); Expected inventory vs. Actual inventory (Retail).

11. Audio_Narrator (Edge TTS)

- **Purpose:** Converts the AI's text output into localized speech using on-device Text-to-Speech (TTS).
- **Architect Settings:**
    - auto_play: Boolean.
    - voice_profile: [Local Dialect 1, Local Dialect 2]
    - playback_speed: [0.75x, 1.0x, 1.25x]

⚡ Category 5: Edge Action Widgets (Hardware & Export)

This is the ultimate differentiator. These widgets allow the AI to interact with the phone's native operating system.

12. Native_Comm_Dispatcher (SMS/Call Intent)

- **Purpose:** Uses Android/iOS intents to draft text messages or initiate phone calls based on the AI's logic.
- **Architect Settings:**
    - action_type: [SMS, WhatsApp_Intent, Phone_Call]
    - target_contact: [Hardcoded Number, or Extracted from Input]
    - payload_template: {{Text_Variables}}
- **Cross-Domain Uses:** Automated emergency SOS (Security); texting a daily order to a supplier (Retail); notifying a parent about a student's grade (Education).

13. Local_Database (Offline Tracker)

- **Purpose:** Silently saves data generated by the app into a local SQLite database or CSV file on the device.
- **Architect Settings:**
    - table_name: (e.g., daily_sales, health_logs)
    - data_schema: Defining the columns to save.
- **Cross-Domain Uses:** Tracking daily crop yields over a month; logging baby feeding

times; maintaining a small business ledger.

14. `Peer_Share_Node` (Zero-Bandwidth Export)

- **Purpose:** Serializes the current state of the app (the UI layout + the data) into a QR code or Bluetooth payload so it can be shared with another device offline.
- **Architect Settings:**
    - `share_method`: [Dense_QR_Code, Nearby_Share/Bluetooth]
    - `allow_editing_on_receive`: Boolean.
- **Cross-Domain Uses:** A teacher sharing a custom quiz to a student's tablet; a site manager sharing a daily safety inspection checklist to workers' phones.

How this elevates your pitch:

With this library, if a judge asks, *"Can this be used for [X Random Industry]?"* your answer is a definitive **yes**.

You can explain that building a localized AI app is no longer about writing Python code; it is simply about snapping together a `VoiceText_Transcriber`, a `LiteRT_Core_Inference`, and a `Local_Database` inside the Bina.ai Studio.